

DeepSDF: Learning Continuous Signed Distance Functions for Shape Representation

Jeong Joon Park^{1,3} Peter Florence^{2,3} Julian Straub³ Richard Newcombe³ Steven Lovegrove³

¹University of Washington ²Massachusetts Institute of Technology ³Facebook Reality Labs

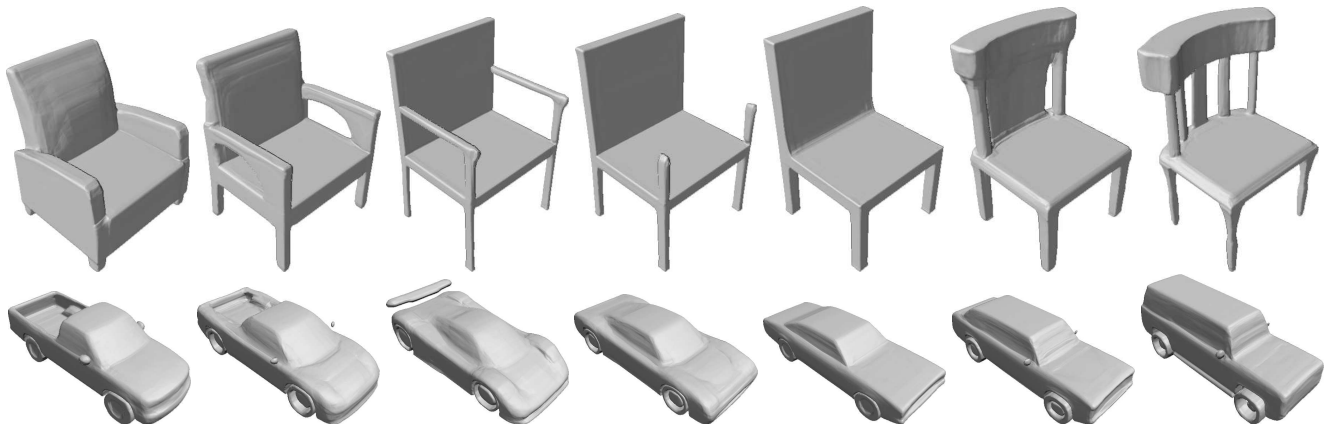


Figure 1: DeepSDF represents signed distance functions (SDFs) of shapes via latent code-conditioned feed-forward decoder networks. Above images are raycast renderings of DeepSDF interpolating between two shapes in the learned shape latent space. Best viewed digitally.

Abstract

Computer graphics, 3D computer vision and robotics communities have produced multiple approaches to representing 3D geometry for rendering and reconstruction. These provide trade-offs across fidelity, efficiency and compression capabilities. In this work, we introduce DeepSDF, a learned continuous Signed Distance Function (SDF) representation of a class of shapes that enables high quality shape representation, interpolation and completion from partial and noisy 3D input data. DeepSDF, like its classical counterpart, represents a shape’s surface by a continuous volumetric field: the magnitude of a point in the field represents the distance to the surface boundary and the sign indicates whether the region is inside (-) or outside (+) of the shape, hence our representation implicitly encodes a shape’s boundary as the zero-level-set of the learned function while explicitly representing the classification of space as being part of the shapes’ interior or not. While classical SDF’s both in analytical or discretized voxel form typically represent the surface of a single shape, DeepSDF can represent an entire class of shapes. Furthermore, we show state-of-the-art performance for learned 3D shape representation and completion while reducing the model size by an order of magnitude compared with previous work.

1. Introduction

Deep convolutional networks which are a mainstay of image-based approaches grow quickly in space and time complexity when directly generalized to the 3rd spatial dimension, and more classical and compact surface representations such as triangle or quad meshes pose problems in training since we may need to deal with an unknown number of vertices and arbitrary topology. These challenges have limited the quality, flexibility and fidelity of deep learning approaches when attempting to either input 3D data for processing or produce 3D inferences for object segmentation and reconstruction.

In this work, we present a novel representation and approach for generative 3D modeling that is efficient, expressive, and fully continuous. Our approach uses the concept of a SDF, but unlike common surface reconstruction techniques which discretize this SDF into a regular grid for evaluation and measurement denoising [14], we instead learn a generative model to produce such a continuous field.

The proposed continuous representation may be intuitively understood as a learned shape-conditioned classifier for which the decision boundary is the surface of the shape itself, as shown in Fig. 2. Our approach shares the generative aspect of other works seeking to map a latent space to a distribution of complex shapes in 3D [52], but critically differs in the central representation. While the notion of an

Work performed while Park and Florence were interns at Facebook.

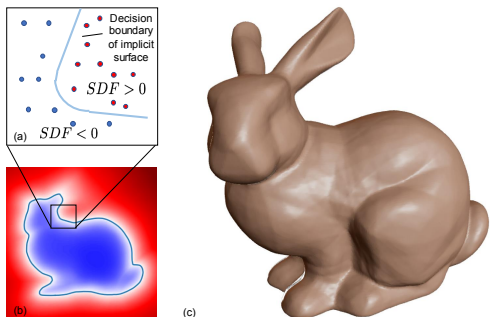


Figure 2: Our DeepSDF representation applied to the Stanford Bunny: (a) depiction of the underlying implicit surface $SDF = 0$ trained on sampled points inside $SDF < 0$ and outside $SDF > 0$ the surface, (b) 2D cross-section of the signed distance field, (c) rendered 3D surface recovered from $SDF = 0$. Note that (b) and (c) are recovered via DeepSDF.

implicit surface defined as a SDF is widely known in the computer vision and graphics communities, to our knowledge no prior works have attempted to directly learn continuous, generalizable 3D generative models of SDFs.

Our contributions include: (i) the formulation of generative shape-conditioned 3D modeling with a continuous implicit surface, (ii) a learning method for 3D shapes based on a probabilistic auto-decoder, and (iii) the demonstration and application of this formulation to shape modeling and completion. Our models produce high quality continuous surfaces with complex topologies, and obtain state-of-the-art results in quantitative comparisons for shape reconstruction and completion. As an example of the effectiveness of our method, our models use only 7.4 MB (megabytes) of memory to represent entire classes of shapes (for example, thousands of 3D chair models) – this is, for example, less than half the memory footprint (16.8 MB) of a single uncompressed 512^3 3D bitmap.

2. Related Work

We review three main areas of related work: 3D representations for shape learning (Sec. 2.1), techniques for learning generative models (Sec. 2.2), and shape completion (Sec. 2.3).

2.1. Representations for 3D Shape Learning

Representations for data-driven 3D learning approaches can be largely classified into three categories: point-based, mesh-based, and voxel-based methods. While some applications such as 3D-point-cloud-based object classification are well suited to these representations, we address their limitations in expressing continuous surfaces with complex topologies.

Point-based. A point cloud is a lightweight 3D representa-

tion that closely matches the raw data that many sensors (i.e. LiDARs, depth cameras) provide, and hence is a natural fit for applying 3D learning. PointNet [36], for example, uses max-pool operations to extract global shape features, and the technique is widely used as an encoder for point generation networks [55, 1]. There is a sizable list of related works for learning on point clouds [37, 51, 56]. A primary limitation, however, of learning with point clouds is that they do not describe topology and are not suitable for producing watertight surfaces.

Mesh-based. Various approaches represent classes of similarly shaped objects, such as morphable human body parts, with predefined template meshes and some of these models demonstrate high fidelity shape generation results [2, 31]. Other recent works [3] use poly-cube mapping [48] for shape optimization. While the use of template meshes is convenient and naturally provides 3D correspondences, it can only model shapes with fixed mesh topology.

Other mesh-based methods use existing [45, 33] or learned [19, 20] parameterization techniques to describe 3D surfaces by morphing 2D planes. The quality of such representations depends on parameterization algorithms that are often sensitive to input mesh quality and cutting strategies. To address this, recent data-driven approaches [55, 19] learn the parameterization task with deep networks. They report, however, that (a) multiple planes are required to describe complex topologies but (b) the generated surface patches are not stitched, i.e. the produced shape is not closed. To generate a closed mesh, sphere parameterization may be used [19, 20], but the resulting shape is limited to the topological sphere. Other works related to learning on meshes propose to use new convolution and pooling operations for meshes [16, 50] or general graphs [8].

Voxel-based. Voxels, which non-parametrically describe volumes with 3D grids of values, are perhaps the most natural extension into the 3D domain of the well-known learning paradigms (i.e., convolution) that have excelled in the 2D image domain. The most straightforward variant of voxel-based learning is to use a dense occupancy grid (occupied / not occupied). Due to the cubically growing compute and memory requirements, however, current methods are only able to handle low resolutions (128^3 or below). As such, voxel-based approaches do not preserve fine shape details [54, 13], and additionally voxels visually appear significantly different than high-fidelity shapes, since when rendered their normals are not smooth. Octree-based methods [49, 41, 22] alleviate the compute and memory limitations of dense voxel methods, extending for example the ability to learn at up to 512^3 resolution [49], but even this resolution is far from producing shapes that are visually compelling.

Aside from occupancy grids, and more closely related to our approach, it is also possible to use a 3D grid of voxels to represent a signed distance function. This inherits

from the success of fusion approaches that use a truncated SDF (TSDF), pioneered in [14, 35], to combine noisy depth maps into a single 3D model. Voxel-based SDF representations have been extensively used for 3D shape learning [57, 15, 46], but their use of discrete voxels is expensive in memory. As a result, these approaches generally present low resolution shapes. Wavelet transform-based methods [27] and dimensionality reduction techniques [27] for distance fields were reported, but they encode the SDF volume of each individual scene rather than a dataset of shapes.

Recently, concurrently to our work, binary implicit surface representations were used by [12, 34], where they train deep networks across classes of shapes to classify 3D points as inside or outside of a shape. Note that the binary occupancy function is a special case of SDF, considering only ‘sign’ of SDF values. As DeepSDF models metric signed distance to the surface, it can be used to raycast against surfaces and compute surface normals with its gradients.

2.2. Representation Learning Techniques

Modern representation learning techniques aim at automatically discovering a set of features that compactly but expressively describe data. For a more extensive review of the field, we refer to Bengio et al. [4].

Generative Adversarial Networks. GANs [18] and their variants [11, 39, 26, 28] learn deep embeddings of target data, from which realistic images are sampled, by training discriminators adversarially against generators. In the 3D domain, Wu et al. [52] trains a GAN to generate objects in a voxel form, while Hamu et al. [20] uses multiple parameterization planes to generate shapes of topological spheres. On the downside, training for GANs is known to be unstable.

Auto-encoders. The ability of auto-encoders as a representation learning tool has been evidenced by the vast variety of 3D shape learning works in the literature [15, 46, 2, 19, 53] who adopt auto-encoders for representation learning. Recent 3D vision works [5, 2, 31] often adopt a variational auto-encoder (VAE) learning scheme, in which bottleneck features are perturbed with Gaussian noise to encourage smooth and complete latent spaces. The regularization on the latent vectors enables exploring the embedding space with gradient descent or random sampling.

Optimizing Latent Vectors. Instead of using the full auto-encoder, an alternative is to learn compact data representations by training *decoder-only* networks. This idea goes back to at least the work of Tan et al. [47] which simultaneously optimizes the latent vectors assigned to each data point and the decoder weights through back-propagation. For inference, an optimal latent vector is searched to match the new observation with fixed decoder parameters. Similar approaches have been extensively studied in [40, 7, 38], for applications including noise reduction, missing measurement completions, and fault detections. Recent approaches

[6, 17] extend the technique by applying deep architectures. Throughout the paper we refer to this class of networks as *auto-decoders*, for they are trained with *self*-reconstruction loss on decoder-only architectures.

2.3. Shape Completion

3D shape completion related works aim to infer unseen parts of the original shape given sparse or partial input observations. This task is analogous to image-inpainting in 2D computer vision.

Classical surface reconstruction methods complete a point cloud into a dense surface by fitting radial basis function (RBF) [9] to approximate implicit surface functions, or by casting the reconstruction from oriented point clouds as a Poisson problem [29]. These methods only model a single shape rather than a dataset.

Various recent methods use data-driven approaches for the 3D completion task. Most of these methods adopt encoder-decoder architectures to reduce partial inputs of occupancy voxels [54], discrete SDF voxels [15], depth maps [42], RGB images [13, 53] or point clouds [46] into a latent vector and subsequently generate a prediction of full volumetric shape based on learned priors.

3. Modeling SDFs with Neural Networks

In this section we present DeepSDF, our continuous shape learning approach. We describe modeling shapes as the zero iso-surface decision boundaries of feed-forward networks trained to represent SDFs. A signed distance function is a continuous function that, for a given spatial point, outputs the point’s distance to the closest surface, whose sign encodes whether the point is inside (negative) or outside (positive) of the watertight surface:

$$SDF(\mathbf{x}) = s : \mathbf{x} \in \mathbb{R}^3, s \in \mathbb{R}. \quad (1)$$

The underlying surface is implicitly represented by the iso-surface of $SDF(\cdot) = 0$. A view of this implicit surface can be rendered through raycasting or rasterization of a mesh obtained with, for example, Marching Cubes [32].

Our key idea is to directly regress the continuous SDF from point samples using deep neural networks. The resulting trained network is able to predict the SDF value of a given query position, from which we can extract the zero level-set surface by evaluating spatial samples. Such surface representation can be intuitively understood as a spatial classifier for which the decision boundary is the surface of the shape itself (Fig. 2). As a universal function approximator [24], deep feed-forward networks in theory can learn continuous SDFs with arbitrary precision. Yet, the precision of the approximation in practice is limited by the finite number of point samples that guide the decision boundaries and the finite capacity of the network due to restricted compute power.

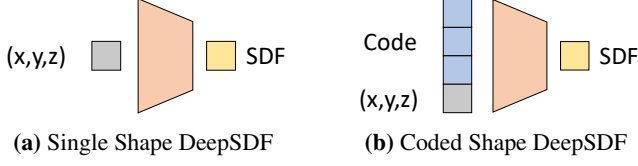


Figure 3: DeepSDF network outputs SDF value at a 3D query location. While (a) the network can memorize a single shape, (b) conditioning the network with a code vector allows DeepSDF to model a large space of shapes, where the shape information is contained in the code vector that is concatenated with the query point.

The most direct application of this approach is to train a single deep network for a given target shape as depicted in Fig. 3a. Given a target shape, we prepare a set of pairs X composed of 3D point samples and their SDF values:

$$X := \{(\mathbf{x}, s) : SDF(\mathbf{x}) = s\}. \quad (2)$$

We train the parameters θ of a multi-layer fully-connected neural network f_θ on the training set S to make f_θ a good approximator of the given SDF in the target domain Ω :

$$f_\theta(\mathbf{x}) \approx SDF(\mathbf{x}), \forall \mathbf{x} \in \Omega. \quad (3)$$

The training is done by minimizing the sum over losses between the predicted and real SDF values of points in X under the following L_1 loss function:

$$\mathcal{L}(f_\theta(\mathbf{x}), s) = |\text{clamp}(f_\theta(\mathbf{x}), \delta) - \text{clamp}(s, \delta)|, \quad (4)$$

where $\text{clamp}(x, \delta) := \min(\delta, \max(-\delta, x))$ introduces the parameter δ to control the distance from the surface over which we expect to maintain a metric SDF. Larger values of δ allow for fast ray-tracing since each sample gives information of safe step sizes [23]. Smaller δ can be used to concentrate network capacity on details near the surface. We used $\delta = 0.1$ in practice. (see supplementary for details)

Once trained, the surface is implicitly represented as the zero iso-surface of $f_\theta(\mathbf{x})$, which can be visualized through raycasting or marching cubes. Another nice property of this approach is that accurate normals can be analytically computed by calculating the spatial derivative $\partial f_\theta(\mathbf{x}) / \partial \mathbf{x}$ via back-propagation through the network.

4. Learning the Latent Space of Shapes

Training a specific neural network for each shape is neither feasible nor very useful. Instead, we want a model that can represent a wide variety of shapes, discover their common properties, and embed them in a low dimensional latent space. To this end, we introduce a latent vector $\mathbf{z}$, which can be thought of as encoding the desired shape, as a second input to the neural network as depicted in Fig. 3b. Conceptually, we map this latent vector to a 3D shape represented

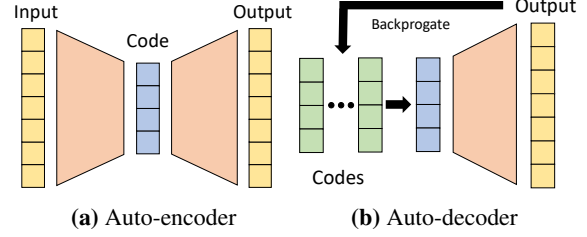


Figure 4: Different from an auto-encoder whose latent code is produced by the encoder, an auto-decoder directly accepts a latent vector as an input. A randomly initialized latent vector is assigned to each data point in the beginning of training, and the latent vectors are optimized along with the decoder weights through standard backpropagation. During inference, decoder weights are fixed, and an optimal latent vector is estimated.

by a continuous SDF. Formally, for some shape indexed by i , f_θ is now a function of a latent code $\mathbf{z}_i$ and a query 3D location $\mathbf{x}$, which outputs the shape’s approximate SDF:

$$f_\theta(\mathbf{z}_i, \mathbf{x}) \approx SDF^i(\mathbf{x}). \quad (5)$$

By conditioning the network output on a latent vector, this formulation allows modeling multiple SDFs with a single neural network. Given the decoding model f_θ , the continuous surface associated with a latent vector $\mathbf{z}$ is similarly represented with the zero iso-surface of $f_\theta(\mathbf{z}, \mathbf{x})$, and the shape can again be discretized for visualization by, for example, raycasting or Marching Cubes.

Throughout the paper we use the coded shape DeepSDF model of Fig. 3b whose decoder is a feed-forward network composed of eight fully connected layers, each of them applied with dropouts. All internal layers are 512-dimensional and have ReLU non-linearities. The output non-linearity regressing the SDF value is tanh. We found training with batch-normalization [25] to be unstable and applied the weight-normalization technique instead [43]. For training, we use the Adam optimizer [30].

In the next section we explain training the decoding model $f_\theta(\mathbf{z}, \mathbf{x})$ and introduce the ‘auto-decoder’ formulation for encoder-less training of shape-coded DeepSDF.

4.1. Motivating Encoder-less Learning

Auto-encoders and encoder-decoder networks are widely used for representation learning as their bottleneck features tend to form natural latent variable representations.

Recently, in applications such as modeling depth maps [5], faces [2], and body shapes [31] a full encoder-decoder network is trained but only the decoder is retained for inference, where they search for an optimal latent vector given some input observation. Since the trained encoder is unused at test time, it is unclear whether (1) training encoder is an effective use of computational resources and (2) it is neces-

sary for researchers to design encoders for various 3D input representations (e.g. points, meshes, octrees, etc).

This motivates us to use an *auto-decoder* for learning a shape embedding without an encoder as depicted in Fig. 4b. We show that learning continuous SDFs with auto-decoder leads to high quality 3D generative models. Further, we develop a probabilistic formulation for training and testing the auto-decoder that naturally introduces latent space regularization for improved generalization. To the best of our knowledge, this work is the first to introduce the auto-decoder learning method to the 3D learning community.

4.2. Auto-decoder-based DeepSDF Formulation

To derive the auto-decoder-based shape-coded DeepSDF formulation we adopt a probabilistic perspective. Given a dataset of N shapes represented with signed distance function $SDF_{i=1}^N$, we prepare a set of K point samples and their signed distance values:

$$X_i = \{(\mathbf{x}_j, s_j) : s_j = SDF^i(\mathbf{x}_j)\}. \quad (6)$$

For an auto-decoder, as there is no encoder, each latent code $\mathbf{z}_i$ is paired with training shape X_i . The posterior over shape code $\mathbf{z}_i$ given the shape SDF samples X_i can be decomposed as:

$$p_\theta(\mathbf{z}_i | X_i) = p(\mathbf{z}_i) \prod_{(\mathbf{x}_j, s_j) \in X_i} p_\theta(s_j | \mathbf{z}_i; \mathbf{x}_j), \quad (7)$$

where θ parameterizes the SDF likelihood. In the latent shape-code space, we assume the prior distribution over codes $p(\mathbf{z}_i)$ to be a zero-mean multivariate-Gaussian with a spherical covariance $\sigma^2 I$. This prior encapsulates the notion that the shape codes should be concentrated and we empirically found it was needed to infer a compact shape manifold and to help converge to good solutions.

In the auto-decoder-based DeepSDF formulation we express the SDF likelihood via a deep feed-forward network $f_\theta(\mathbf{z}_i, \mathbf{x}_j)$ and, without loss of generality, assume that the likelihood takes the form:

$$p_\theta(s_j | \mathbf{z}_i; \mathbf{x}_j) = \exp(-\mathcal{L}(f_\theta(\mathbf{z}_i, \mathbf{x}_j), s_j)). \quad (8)$$

The SDF prediction $\tilde{s}_j = f_\theta(\mathbf{z}_i, \mathbf{x}_j)$ is represented using a fully-connected network. $\mathcal{L}(\tilde{s}_j, s_j)$ is a loss function penalizing the deviation of the network prediction from the actual SDF value s_j . One example for the cost function is the standard L_2 loss function which amounts to assuming Gaussian noise on the SDF values. In practice we use the clamped L_1 cost from Eq. 4 for reasons outlined previously.

At training time we maximize the joint log posterior over all training shapes with respect to the individual shape codes $\{\mathbf{z}_i\}_{i=1}^N$ and the network parameters θ :

$$\arg \min_{\theta, \{\mathbf{z}_i\}_{i=1}^N} \sum_{i=1}^N \left(\sum_{j=1}^K \mathcal{L}(f_\theta(\mathbf{z}_i, \mathbf{x}_j), s_j) + \frac{1}{\sigma^2} \|\mathbf{z}_i\|_2^2 \right). \quad (9)$$

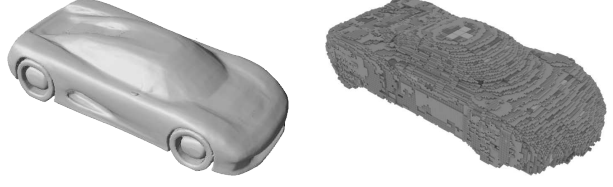


Figure 5: Compared to car shapes memorized using OGN [49] (right), our models (left) preserve details and render visually pleasing results as DeepSDF provides oriented surface normals.

At inference time, after training and fixing θ , a shape code $\mathbf{z}_i$ for shape X_i can be estimated via Maximum-a-Posterior (MAP) estimation as:

$$\hat{\mathbf{z}} = \arg \min_{\mathbf{z}} \sum_{(\mathbf{x}_j, s_j) \in X} \mathcal{L}(f_\theta(\mathbf{z}, \mathbf{x}_j), s_j) + \frac{1}{\sigma^2} \|\mathbf{z}\|_2^2. \quad (10)$$

Crucially, this formulation is valid for SDF samples X of arbitrary size and distribution because the gradient of the loss with respect to $\mathbf{z}$ can be computed separately for each SDF sample. This implies that DeepSDF can handle any form of partial observations such as depth maps. This is a major advantage over the auto-encoder framework whose encoder expects a test input similar to the training data, e.g. shape completion networks of [15, 56] require preparing training data of partial shapes.

To incorporate the latent shape code, we stack the code vector and the sample location as depicted in Fig. 3b and feed it into the same fully-connected NN described previously at the input layer and additionally at the 4th layer. We again use the Adam optimizer [30]. The latent vector $\mathbf{z}$ is initialized randomly from $\mathcal{N}(0, 0.01^2)$.

Note that while both VAE and the proposed auto-decoder formulation share the zero-mean Gaussian prior on the latent codes, we found that the stochastic nature of the VAE optimization did not lead to good training results.

5. Data Preparation

To train our continuous SDF model, we prepare the SDF samples X (Eq. 2) for each mesh, which consists of 3D points and their SDF values. While SDF can be computed through a distance transform for any watertight shapes from real or synthetic data, we train with synthetic objects, (e.g. ShapeNet [10]), for which we are provided complete 3D shape meshes. To prepare data, we start by normalizing each mesh to a unit sphere and sampling 500,000 spatial points $\mathbf{x}$'s: we sample more aggressively near the surface of the object as we want to capture a more detailed SDF near the surface. For an ideal oriented watertight mesh, computing the signed distance value of $\mathbf{x}$ would only involve finding the closest triangle, but we find that human designed meshes are commonly not watertight and contain undesired internal structures. To obtain the *shell* of a

Method	Type	Discretization	Complex topologies	Closed surfaces	Surface normals	Model size (GB)	Inf. time (s)	Eval. tasks
3D-EPN [15]	Voxel SDF	32^3 voxels	✓	✓	✓	0.42	-	C
OGN [49]	Octree	256^3 voxels	✓	✓		0.54	0.32	K
AtlasNet-Sphere [19]	Parametric mesh	1 patch		✓		0.015	0.01	K, U
AtlasNet-25 [19]	Parametric mesh	25 patches	✓			0.172	0.32	K, U
DeepSDF (ours)	Continuous SDF	none	✓	✓	✓	0.0074	9.72	K, U, C

Table 1: Overview of the benchmarked methods. AtlasNet-Sphere can only describe topological-spheres, voxel/octree occupancy methods (i.e. OGN) only provide 8 directions for normals, and AtlasNet does not provide oriented normals. Our tasks evaluated are: (K) representing known shapes, (U) representing unknown shapes, and (C) shape completion.

mesh with proper orientation, we set up equally spaced virtual cameras around the object, and densely sample surface points, denoted P_s , with surface normals oriented towards the camera. Double sided triangles visible from both orientations (indicating that the shape is not closed) cause problems in this case, so we discard mesh objects containing too many of such faces. Then, for each x , we find the closest point in P_s , from which the $SDF(x)$ can be computed. We refer readers to supplementary material for further details.

6. Results

We conduct a number of experiments to show the representational power of DeepSDF, both in terms of its ability to describe geometric details and its generalization capability to learn a desirable shape embedding space. Largely, we propose four main experiments designed to test its ability to 1) represent training data, 2) use learned feature representation to reconstruct unseen shapes, 3) apply shape priors to complete partial shapes, and 4) learn smooth and complete shape embedding space from which we can sample new shapes. For all experiments we use the popular ShapeNet [10] dataset.

We select a representative set of 3D learning approaches to comparatively evaluate aforementioned criteria: a recent octree-based method (OGN) [49], a mesh-based method (AtlasNet) [19], and a volumetric SDF-based shape completion method (3D-EPN) [15] (Table 1). These works show state-of-the-art performance in their respective representations and tasks, so we omit comparisons with the works that have already been compared: e.g. OGN’s octree model outperforms regular voxel approaches, while AtlasNet compares itself with various points, mesh, or voxel based methods and 3D-EPN with various completion methods.

6.1. Representing Known 3D Shapes

First, we evaluate the capacity of the model to represent *known* shapes, i.e. shapes that were in the training set, from only a restricted-size latent code — testing the limit of expressive capability of the representations.

Method \ metric	CD, mean	CD, median	EMD, mean	EMD, median
OGN	0.167	0.127	0.043	0.042
AtlasNet-Sph.	0.210	0.185	0.046	0.045
AtlasNet-25	0.157	0.140	0.060	0.060
DeepSDF	0.084	0.058	0.043	0.042

Table 2: Comparison for representing known shapes (K) for cars trained on ShapeNet. CD = Chamfer Distance (30,000 points) multiplied by 10^3 , EMD = Earth Mover’s Distance (500 points).

Quantitative comparison in Table 2 shows that the proposed DeepSDF significantly beats OGN and AtlasNet in Chamfer distance against the true shape computed with a large number of points (30,000). The difference in earth mover distance (EMD) is smaller because 500 points do not well capture the additional precision. Figure 5 shows a qualitative comparison of DeepSDF to OGN.

6.2. Representing Test 3D Shapes (auto-encoding)

For encoding *unknown* shapes, i.e. shapes in the held-out test set, DeepSDF again significantly outperforms AtlasNet on a wide variety of shape classes and metrics as shown in Table 3. Note that AtlasNet performs reasonably well at classes of shapes that have mostly consistent topology without holes (like planes) but struggles more on classes that commonly have holes, like chairs. This is shown in Fig. 6 where AtlasNet fails to represent the fine detail of the back of the chair. Figure 7 shows more examples of detailed reconstructions on test data from DeepSDF as well as two example failure cases.

6.3. Shape Completion

A major advantage of the proposed DeepSDF approach for representation learning is that inference can be performed from an arbitrary number of SDF samples. In the DeepSDF framework, shape completion amounts to solving for the shape code that best explains a partial shape observation via Eq. 10. Given the shape-code a complete shape can be rendered using the priors encoded in the decoder.

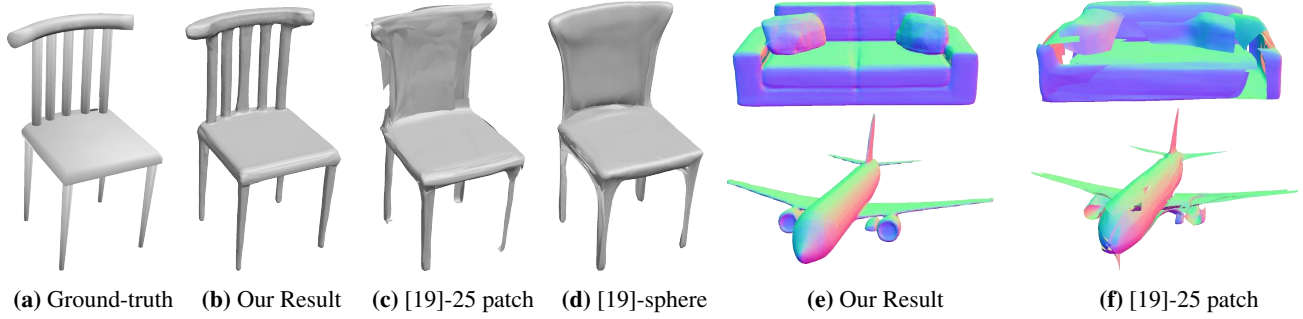


Figure 6: Reconstruction comparison between DeepSDF and AtlasNet [19] (with 25-plane and sphere parameterization) for test shapes. Note that AtlasNet fails to capture the fine details of the chair, and that (f) shows holes on the surface of sofa and the plane.



Figure 7: Reconstruction of test shapes. From left to right alternating: ground truth shape and our reconstruction. The two right most columns show failure modes of DeepSDF. These failures are likely due to lack of training data and failure of minimization convergence.

CD, mean	chair	plane	table	lamp	sofa
AtlasNet-Sph.	0.752	0.188	0.725	2.381	0.445
AtlasNet-25	0.368	0.216	0.328	1.182	0.411
DeepSDF	0.204	0.143	0.553	0.832	0.132
CD, median					
AtlasNet-Sph.	0.511	0.079	0.389	2.180	0.330
AtlasNet-25	0.276	0.065	0.195	0.993	0.311
DeepSDF	0.072	0.036	0.068	0.219	0.088
EMD, mean					
AtlasNet-Sph.	0.071	0.038	0.060	0.085	0.050
AtlasNet-25	0.064	0.041	0.073	0.062	0.063
DeepSDF	0.049	0.033	0.050	0.059	0.047
Mesh acc., mean					
AtlasNet-Sph.	0.033	0.013	0.032	0.054	0.017
AtlasNet-25	0.018	0.013	0.014	0.042	0.017
DeepSDF	0.009	0.004	0.012	0.013	0.004

Table 3: Comparison for representing unknown shapes (U) for various classes of ShapeNet. Mesh accuracy as defined in [44] is the minimum distance d such that 90% of generated points are within d of the ground truth mesh. Lower is better for all metrics.

We test our completion scheme using single view depth observations which is a common use-case and maps well to our architecture without modification. Note that we currently require the depth observations in the canonical shape frame of reference.

To generate SDF point samples from the depth image observation, we sample two points for each depth observation, each of them located η distance away from the measured

Method \\Metric	<i>lower is better</i>				<i>higher is better</i>	
	CD, med.	CD, mean	EMD	Mesh acc.	Mesh comp.	Cos sim.
chair						
3D-EPN	2.25	2.83	0.084	0.059	0.209	0.752
DeepSDF	1.28	2.11	0.071	0.049	0.500	0.766
plane						
3D-EPN	1.63	2.19	0.063	0.040	0.165	0.710
DeepSDF	0.37	1.16	0.049	0.032	0.722	0.823
sofa						
3D-EPN	2.03	2.18	0.071	0.049	0.254	0.742
DeepSDF	0.82	1.59	0.059	0.041	0.541	0.810

Table 4: Comparison for shape completion (C) from partial range scans of unknown shapes from ShapeNet.

surface point (along surface normal estimate). With small η we approximate the signed distance value of those points to be η and $-\eta$, respectively. We solve for Eq. 10 with loss function of Eq. 4 using clamp value of η . Additionally, we incorporate free-space observations, (i.e. empty-space between surface and camera), by sampling points along the freespace-direction and enforce larger-than-zero constraints. The freespace loss is $|f_\theta(z, x_j)|$ if $f_\theta(z, x_j) < 0$ and 0 otherwise.

Given the SDF point samples and empty space points, we similarly optimize the latent vector using MAP estimation. Tab. 4 and Figs. (8, 9) respectively shows quantitative and qualitative shape completion results. Compared to one of the most recent completion approaches [15] using volu-

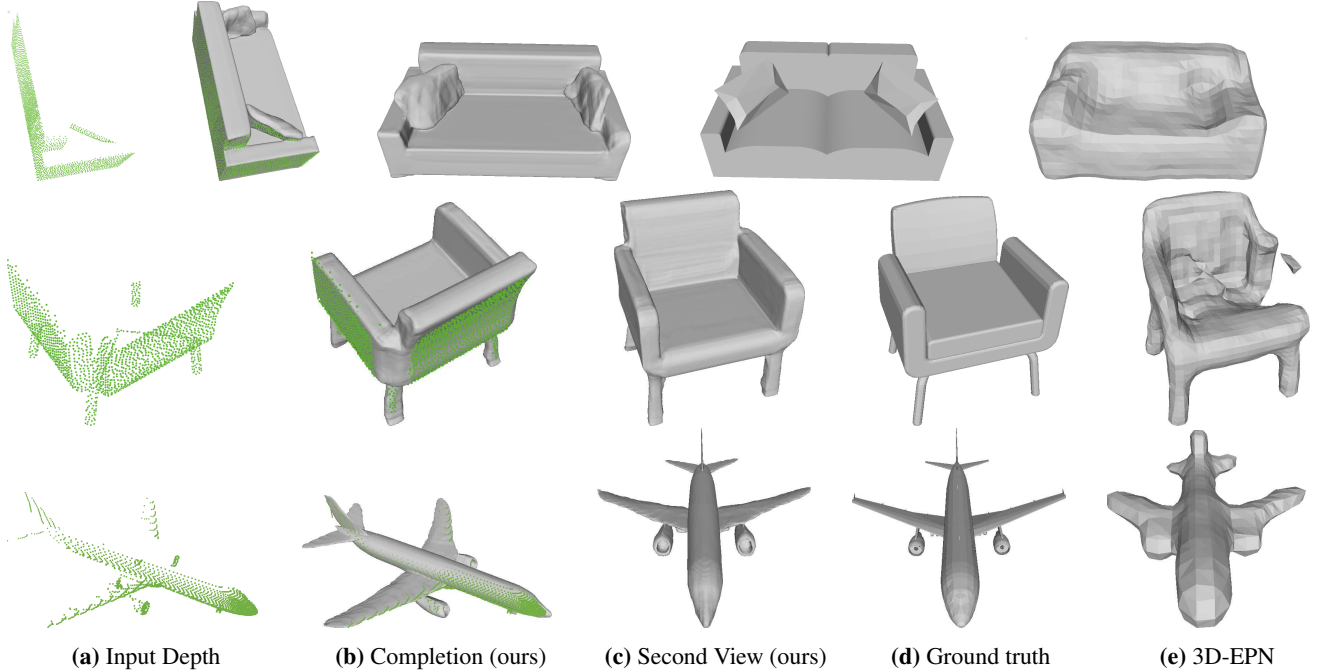


Figure 8: For a given depth image visualized as a green point cloud, we show a comparison of shape completions from our DeepSDF approach against the true shape and 3D-EPN.

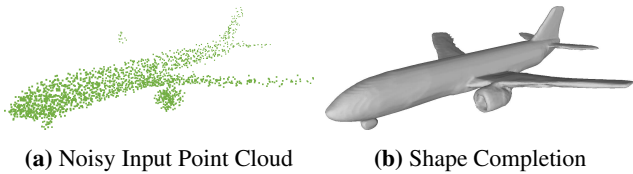


Figure 9: Demonstration of DeepSDF shape completion from a partial noisy point cloud. Input here is generated by perturbing the 3D point cloud positions generated by the ground truth depth map by 1.5% of the plane length. We provide a comprehensive analysis of robustness to noise in the supplementary material.

metric shape representation, our continuous SDF approach produces more visually pleasing and accurate shape reconstructions. While a few recent shape completion methods were presented [21, 53], we could not find the code to run the comparisons, and their underlying 3D representation is voxel grid which we extensively compare against.

6.4. Latent Space Shape Interpolation

To show that our learned shape embedding is complete and continuous, we render the results of the decoder when a pair of shapes are interpolated in the latent vector space (Fig. 1). The results suggests that the embedded continuous SDF’s are of meaningful shapes and that our representation extracts common interpretable shape features, such as the arms of a chair, that interpolate linearly in the latent space.

7. Conclusion & Future Work

DeepSDF significantly outperforms the applicable benchmarked methods across shape representation and completion tasks and simultaneously addresses the goals of representing complex topologies, closed surfaces, while providing high quality surface normals of the shape. However, while point-wise forward sampling of a shape’s SDF is efficient, shape completion (auto-decoding) takes considerably more time during inference due to the need for explicit optimization over the latent vector. We look to increase performance by replacing ADAM optimization with more efficient Gauss-Newton or similar methods that make use of the analytic derivatives of the model.

DeepSDF models enable representation of more complex shapes without discretization errors with significantly less memory than previous state-of-the-art results as shown in Table 1, demonstrating an exciting route ahead for 3D shape learning. The clear ability to produce quality latent shape space interpolation opens the door to reconstruction algorithms operating over scenes built up of such efficient encodings. However, DeepSDF currently assumes models are in a canonical pose and as such completion in-the-wild requires explicit optimization over a $SE(3)$ transformation space increasing inference time. Finally, to represent the true *space-of-possible-scenes* including dynamics and textures in a single embedding remains a major challenge, one which we continue to explore.

References

- [1] P. Achlioptas, O. Diamanti, I. Mitliagkas, and L. Guibas. Learning representations and generative models for 3d point clouds. 2018.
- [2] T. Bagautdinov, C. Wu, J. Saragih, P. Fua, and Y. Sheikh. Modeling facial geometry using compositional vaes. 1:1.
- [3] P. Baqué, E. Remelli, F. Fleuret, and P. Fua. Geodesic convolutional shape optimization. *arXiv preprint arXiv:1802.04016*, 2018.
- [4] Y. Bengio, A. Courville, and P. Vincent. Representation learning: A review and new perspectives. *TPAMI*, 35(8):1798–1828, 2013.
- [5] M. Bloesch, J. Czarowski, R. Clark, S. Leutenegger, and A. J. Davison. Codeslam-learning a compact, optimisable representation for dense visual slam. *arXiv preprint arXiv:1804.00874*, 2018.
- [6] P. Bojanowski, A. Joulin, D. Lopez-Pas, and A. Szlam. Optimizing the latent space of generative networks. In J. Dy and A. Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 600–609. PMLR, 10–15 Jul 2018.
- [7] M. Bouakkaz and M.-F. Harkat. Combined input training and radial basis function neural networks based nonlinear principal components analysis model applied for process monitoring. In *IJCCI*, pages 483–492, 2012.
- [8] J. Bruna, W. Zaremba, A. Szlam, and Y. LeCun. Spectral networks and locally connected networks on graphs. *arXiv preprint arXiv:1312.6203*, 2013.
- [9] J. C. Carr, R. K. Beatson, J. B. Cherrie, T. J. Mitchell, W. R. Fright, B. C. McCallum, and T. R. Evans. Reconstruction and representation of 3d objects with radial basis functions. In *SIGGRAPH*, pages 67–76. ACM, 2001.
- [10] A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su, et al. Shapenet: An information-rich 3d model repository. *arXiv preprint arXiv:1512.03012*, 2015.
- [11] X. Chen, Y. Duan, R. Houthoofd, J. Schulman, I. Sutskever, and P. Abbeel. Infogan: Interpretable representation learning by information maximizing generative adversarial nets. In *NIPS*, pages 2172–2180, 2016.
- [12] Z. Chen and H. Zhang. Learning implicit fields for generative shape modeling. *arXiv preprint arXiv:1812.02822*, 2018.
- [13] C. B. Choy, D. Xu, J. Gwak, K. Chen, and S. Savarese. 3d-r2n2: A unified approach for single and multi-view 3d object reconstruction. In *ECCV*, pages 628–644. Springer, 2016.
- [14] B. Curless and M. Levoy. A volumetric method for building complex models from range images. In *SIGGRAPH*, pages 303–312. ACM, 1996.
- [15] A. Dai, C. Ruizhongtai Qi, and M. Niessner. Shape completion using 3d-encoder-predictor cnns and shape synthesis. In *CVPR*, pages 5868–5877, 2017.
- [16] M. Defferrard, X. Bresson, and P. Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. In *NIPS*, pages 3844–3852, 2016.
- [17] J. Fan and J. Cheng. Matrix completion by deep matrix factorization. *Neural Networks*, 98:34–41, 2018.
- [18] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial nets. In *NIPS*, pages 2672–2680, 2014.
- [19] T. Groueix, M. Fisher, V. G. Kim, B. C. Russell, and M. Aubry. Atlasnet: A papier-mâché approach to learning 3d surface generation. *arXiv preprint arXiv:1802.05384*, 2018.
- [20] H. B. Hamu, H. Maron, I. Kezurer, G. Avineri, and Y. Lipman. Multi-chart generative surface modeling. *arXiv preprint arXiv:1806.02143*, 2018.
- [21] X. Han, Z. Li, H. Huang, E. Kalogerakis, and Y. Yu. High-resolution shape completion using deep neural networks for global structure and local geometry inference.
- [22] C. Häne, S. Tulsiani, and J. Malik. Hierarchical surface prediction for 3d object reconstruction. In *3D Vision (3DV), 2017 International Conference on*, pages 412–420. IEEE, 2017.
- [23] J. C. Hart. Sphere tracing: A geometric method for the antialiased ray tracing of implicit surfaces. *The Visual Computer*, 12(10):527–545, 1996.
- [24] K. Hornik, M. Stinchcombe, and H. White. Multilayer feed-forward networks are universal approximators. *Neural networks*, 2(5):359–366, 1989.
- [25] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015.
- [26] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros. Image-to-image translation with conditional adversarial networks. *arXiv preprint*, 2017.
- [27] M. W. Jones. Distance field compression. *Journal of WSCG*, 12(2):199–204, 2004.
- [28] T. Karras, T. Aila, S. Laine, and J. Lehtinen. Progressive growing of gans for improved quality, stability, and variation. *arXiv preprint arXiv:1710.10196*, 2017.
- [29] M. Kazhdan and H. Hoppe. Screened poisson surface reconstruction. *ACM TOG*, 32(3):29, 2013.
- [30] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [31] O. Litany, A. Bronstein, M. Bronstein, and A. Makadia. Deformable shape completion with graph convolutional autoencoders. *CVPR*, 2017.
- [32] W. E. Lorensen and H. E. Cline. Marching cubes: A high resolution 3d surface construction algorithm. In *SIGGRAPH*, volume 21, pages 163–169. ACM, 1987.
- [33] H. Maron, M. Galun, N. Aigerman, M. Trope, N. Dym, E. Yumer, V. G. Kim, and Y. Lipman. Convolutional neural networks on surfaces via seamless toric covers. 2017.
- [34] L. Mescheder, M. Oechsle, M. Niemeyer, S. Nowozin, and A. Geiger. Occupancy networks: Learning 3d reconstruction in function space. *arXiv preprint arXiv:1812.03828*, 2018.
- [35] R. A. Newcombe, S. Izadi, O. Hilliges, D. Molyneaux, D. Kim, A. J. Davison, P. Kohi, J. Shotton, S. Hodges, and A. Fitzgibbon. Kinectfusion: Real-time dense surface mapping and tracking. In *ISMAR*, pages 127–136. IEEE, 2011.
- [36] C. R. Qi, H. Su, K. Mo, and L. J. Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *CVPR*, pages 652–660, 2017.

- [37] C. R. Qi, L. Yi, H. Su, and L. J. Guibas. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. In *NIPS*, pages 5099–5108, 2017.
- [38] Z. Qunxiong and L. Chengfei. Dimensionality reduction with input training neural network and its application in chemical process modelling. *Chinese Journal of Chemical Engineering*, 14(5):597–603, 2006.
- [39] A. Radford, L. Metz, and S. Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434*, 2015.
- [40] V. Reddy and M. Mavrouniotis. An input-training neural network approach for gross error detection and sensor replacement. *Chemical Engineering Research and Design*, 76(4):478–489, 1998.
- [41] G. Riegler, A. O. Ulusoy, and A. Geiger. Octnet: Learning deep 3d representations at high resolutions. In *CVPR*, pages 6620–6629. IEEE, 2017.
- [42] J. Rock, T. Gupta, J. Thorsen, J. Gwak, D. Shin, and D. Hoiem. Completing 3d object shape from one depth image. In *CVPR*, pages 2484–2493, 2015.
- [43] T. Salimans and D. P. Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. In *NIPS*, pages 901–909, 2016.
- [44] S. M. Seitz, B. Curless, J. Diebel, D. Scharstein, and R. Szeliski. A comparison and evaluation of multi-view stereo reconstruction algorithms. pages 519–528. IEEE, 2006.
- [45] A. Sinha, J. Bai, and K. Ramani. Deep learning 3d shape surfaces using geometry images. In *ECCV*, pages 223–240. Springer, 2016.
- [46] D. Stutz and A. Geiger. Learning 3d shape completion from laser scan data with weak supervision. In *CVPR*, pages 1955–1964, 2018.
- [47] S. Tan and M. L. Mayrovouniotis. Reducing data dimensionality through optimizing neural network inputs. *AIChE Journal*, 41(6):1471–1480, 1995.
- [48] M. Tarini, K. Hormann, P. Cignoni, and C. Montani. Polycube-maps. In *ACM TOG*, volume 23, pages 853–860. ACM, 2004.
- [49] M. Tatarchenko, A. Dosovitskiy, and T. Brox. Octree generating networks: Efficient convolutional architectures for high-resolution 3d outputs. In *ICCV*, 2017.
- [50] N. Verma, E. Boyer, and J. Verbeek. Feastnet: Feature-steered graph convolutions for 3d shape analysis.
- [51] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon. Dynamic graph cnn for learning on point clouds. *arXiv preprint arXiv:1801.07829*, 2018.
- [52] J. Wu, C. Zhang, T. Xue, B. Freeman, and J. Tenenbaum. Learning a probabilistic latent space of object shapes via 3d generative-adversarial modeling. In *NIPS*, pages 82–90, 2016.
- [53] J. Wu, C. Zhang, X. Zhang, Z. Zhang, W. T. Freeman, and J. B. Tenenbaum. Learning shape priors for single-view 3d completion and reconstruction. *arXiv preprint arXiv:1809.05068*, 2018.
- [54] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao. 3d shapenets: A deep representation for volumetric shapes. In *CVPR*, pages 1912–1920, 2015.
- [55] Y. Yang, C. Feng, Y. Shen, and D. Tian. Foldingnet: Interpretable unsupervised learning on 3d point clouds. *arXiv preprint arXiv:1712.07262*, 2017.
- [56] W. Yuan, T. Khot, D. Held, C. Mertz, and M. Hebert. Pcn: Point completion network. In *3DV*, 2018.
- [57] A. Zeng, S. Song, M. Nießner, M. Fisher, J. Xiao, and T. Funkhouser. 3dmatch: Learning local geometric descriptors from rgb-d reconstructions. In *CVPR*, pages 199–208. IEEE, 2017.